DSA Lecture 3

Linked Lists: Deletion, Complexity, Tail Optimization,
Circular and Doubly Linked Lists

Study notes reconstructed from the supplied 106-page lecture transcript

Implementation language: Java · Beginner-friendly explanations · Algorithms, code, dry runs and revision

Source-first note: these notes follow the topics and explanations supported by the transcript. Where the transcript describes code verbally or is unclear, the implementation below is a clean illustrative reconstruction, not a verbatim copy of the instructor's project.

Contents

• 1. Lecture overview and learning outcomes

• 2. Singly linked list: deleting the last node

• 3. Special cases and exception handling

• 4. Time complexity of linked-list operations

• 5. Optimization with a tail pointer

• 6. Singly circular linked list

• 7. Doubly linked list

• 8. Circular doubly linked list

• 9. Implementation patterns and dry runs

• 10. Common mistakes

• 11. Practice questions and answer key

• 12. Final revision checklist

1. Lecture overview and learning outcomes

The lecture begins by revisiting the singly linked list built in the previous session, especially delete-last. It then discusses operation complexity, improving add-last by maintaining a tail reference, and implementing circular and doubly linked list variants. The instructor repeatedly emphasizes drawing pointer diagrams first, coding from those diagrams, and checking edge cases.

By the end, you should be able to

• Explain why deleting the last node of a singly linked list requires reaching the second-last node.

• Handle empty-list and single-node cases safely.

• Estimate the time complexity of common linked-list operations.

• Explain why a tail pointer improves add-last but does not make delete-last constant time in a singly linked list.

• Distinguish linear, circular singly, doubly, and circular doubly linked lists.

• Update next and previous links without losing part of the list.

Lecture sequence at a glance

Section	Core idea
Delete last	Keep a trailing pointer to the second-last node; set its next to null.
Complexity	Traversal-based operations generally grow with the number of nodes.
Tail pointer	A direct reference to the last node makes add-last O(1).
Circular singly list	The last node points back to head; traversal stops when it returns to head.
Doubly list	Each node has next and previous links; both directions must remain consistent.
Circular doubly list	Forward and backward links form a loop; head/tail boundaries are connected.

2. Singly linked list: deleting the last node

A singly linked-list node contains data and a reference named next. The head reference points to the first node. The last node is identified by next == null.

```
head
 |
 v
[10 | next] -> [20 | next] -> [30 | null]
                                  ^
                              last node
```

Why two pointers?

To remove the last node, the algorithm needs both the last node and the node before it. The last node is easy to find by traversal, but a singly linked list cannot move backward. The lecture uses two references: trav moves forward, while temp trails one node behind.

```
Node temp = null;
Node trav = head;

while (trav.next != null) {
    temp = trav;
    trav = trav.next;
}
// trav is last; temp is second-last (for lists with >= 2 nodes)
temp.next = null;
```

Dry run: 10 -> 20 -> 30

Iteration	temp	trav	Meaning
Start	null	10	Both initialized
1	10	20	temp trails behind
2	20	30	trav is at last node
After loop	20	30	Set temp.next = null

After temp.next = null, the list is 10 -> 20 -> null. The old last node is no longer reachable from head, so Java's garbage collector can reclaim it when appropriate.

> Important: the code above is only the general case. It must not run unchanged for an empty list or a list containing one node.

3. Special cases and exception handling

Case A: empty list

If head == null, there is no last node to delete. The transcript handles this by throwing a RuntimeException with a message such as "List is empty".

Case B: exactly one node

If head.next == null, head is both the first and last node. There is no second-last node, so temp remains null. The correct action is head = null.

Complete illustrative Java implementation

```
public class SinglyLinkedList {
    private static class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
        }
    }

    private Node head;

    public void addLast(int value) {
        Node node = new Node(value);
        if (head == null) {
            head = node;
            return;
        }

        Node trav = head;
        while (trav.next != null) {
            trav = trav.next;
        }
        trav.next = node;
    }

    public void deleteLast() {
        if (head == null) {
            throw new RuntimeException("List is empty");
        }

        // One node: removing it makes the list empty.
        if (head.next == null) {
            head = null;
            return;
        }

        Node temp = null;
        Node trav = head;

        while (trav.next != null) {
            temp = trav;
            trav = trav.next;
        }

        // temp is the second-last node.
        temp.next = null;
    }

    public void display() {
        Node trav = head;
        while (trav != null) {
            System.out.print(trav.data + " -> ");
            trav = trav.next;
        }
        System.out.println("null");
    }

    public static void main(String[] args) {
        SinglyLinkedList list = new SinglyLinkedList();
        list.addLast(10);
        list.addLast(20);
        list.addLast(30);
        list.display();       // 10 -> 20 -> 30 -> null

        list.deleteLast();
        list.display();       // 10 -> 20 -> null

        list.deleteLast();
        list.deleteLast();
        list.display();       // null

        // Uncomment to test the empty-list exception:
        // list.deleteLast();
```

```
    }
}
```

Why does the single-node case need a separate branch?

For a one-node list, head.next is null from the beginning. If the general algorithm is used, the loop does not execute and temp stays null. Attempting temp.next = null would throw NullPointerException. Setting head = null handles the case correctly.

4. Time complexity of linked-list operations

Let n be the number of nodes. Complexity describes how the work grows as the list grows. The lecture advises learning the complexity of each operation, not only memorizing linked-list definitions.

Operation	Typical implementation	Time	Reason
Display / traversal	Visit every node	O(n)	Work grows with node count.
Add first	Create node and redirect head	O(1)	A fixed number of reference updates.
Add last, head only	Traverse to final node, attach new node	O(n)	Must locate the last node.
Add last, with tail	Attach through tail; update tail	O(1)	Last node is already available.
Delete first	Move head to head.next	O(1)	No traversal needed.
Delete last, singly linked	Find second-last node	O(n)	Must traverse from head.
Insert/delete at position	Traverse to the relevant location	O(n) worst case	The target may be near the end.
Delete all	Depends on implementation/API	Often O(n) for manual unlinking	Each node may need to be visited.

Position-based complexity

If an operation traverses to a position p, its work can be described as O(p). When discussing growth against the total number of nodes, the worst case is O(n); a typical position may also require linear work. Constant factors such as n/2 are omitted in Big-O, so O(n/2) simplifies to O(n).

> Exam habit from the lecture: if a question does not specify best, average, or worst case, read the wording carefully. The lecturer says average case is generally used in that situation; follow the convention expected by your course or question.

5. Optimization with a tail pointer

A basic singly linked list stores head. To add at the end, it traverses from head until next == null, which takes O(n). If adding at the end is frequent, maintain tail as a reference to the last node.

```
head                          tail
 |                              |
 v                              v
[10 | next] -> [20 | next] -> [30 | null]
```

Add-last with tail

```
Node newNode = new Node(value);

if (head == null) {
    head = newNode;
    tail = newNode;
} else {
    tail.next = newNode;
    tail = newNode;
}
```

For a non-empty list, two reference updates are enough: old tail.next = newNode, then tail = newNode. This changes add-last from O(n) to O(1).

The invariant you must preserve

At all times, tail must point to the actual last node. When the list is empty, both head and tail should be null. When the first node is inserted, both should point to that node.

Why tail does not make delete-last O(1)

A tail pointer tells us which node is last, but it does not tell us which node is second-last. In a singly linked list, links only point forward. Deleting the last node still requires traversal to the second-last node, so the operation remains O(n).

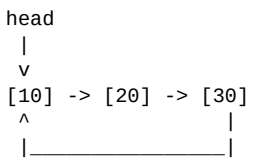
Operation	Head only	Head + tail
Add first	O(1)	O(1), with tail update when initially empty
Add last	O(n)	O(1)
Delete first	O(1)	O(1), update tail if list becomes empty
Delete last	O(n)	O(n)
Extra bookkeeping	Less	Must keep tail correct

Implementation note

When adding or deleting nodes, update every reference that represents the list's state. In particular: adding the first node updates head and tail; deleting the only node clears both; deleting the last node must move tail to the second-last node.

6. Singly circular linked list

In a circular singly linked list, the last node's next points to head instead of null. There is no null terminator in a non-empty circular list.



Traversal rule

A typical linear list traversal stops at null. A circular list must stop when the traversal returns to head; otherwise it loops forever.

```
if (head != null) {
    Node trav = head;
    do {
        System.out.print(trav.data + " ");
        trav = trav.next;
    } while (trav != head);
}
```

```
System.out.println();
```

The do-while pattern is useful because a non-empty circular list must print its head node before testing whether the traversal has returned to head. Always check head != null first.

Add first and add last: pointer logic

• Empty list: the new node points to itself; head points to that node.

• Non-empty circular list: find the last node whose next is head.

• Add first: newNode.next = head; update the old last node's next to newNode; then move head to newNode.

• Add last: last.next = newNode; newNode.next = head.

If the implementation also maintains tail, insertion at the end can be O(1), provided tail.next always equals head. The transcript spends time tracing the loop condition and the point where traversal returns to head.

Common circular-list mistakes

• Using trav != null as the loop condition: the traversal never reaches null in a valid circular list.

• Forgetting to reconnect the last node to the new head after add-first.

• Failing to make the new node point to itself when inserting into an empty list.

• Not updating head when deleting the head node.

7. Doubly linked list

A doubly linked-list node stores data, next, and previous references. The next link leads forward; previous leads backward. In a conventional non-circular doubly linked list, head.previous is null and tail.next is null.

```
null <- [10] <-> [20] <-> [30] -> null
         head                 tail
```

Node structure

```
private static class Node {
    int data;
    Node next;
    Node previous;

    Node(int data) {
        this.data = data;
    }
}
```

Add-last pointer updates

```
Node newNode = new Node(value);

if (head == null) {
    head = newNode;
    tail = newNode;
} else {
    tail.next = newNode;
    newNode.previous = tail;
    tail = newNode;
}
```

The important extra step compared with a singly linked list is newNode.previous = tail. Both directions must agree: if A.next is B, then B.previous should be A.

Add-first pointer updates

```
Node newNode = new Node(value);

if (head == null) {
    head = newNode;
    tail = newNode;
} else {
    newNode.next = head;
    head.previous = newNode;
    head = newNode;
}
```

Delete-first pointer updates

```
if (head == null) {
    throw new RuntimeException("List is empty");
}
if (head == tail) {       // exactly one node
    head = null;
    tail = null;
} else {
    head = head.next;
    head.previous = null;
}
```

Delete-last pointer updates

```
if (head == null) {
    throw new RuntimeException("List is empty");
}
if (head == tail) {
    head = null;
    tail = null;
} else {
    tail = tail.previous;
    tail.next = null;
}
```

With both head and tail references, deleting the last node of a doubly linked list is O(1): tail.previous directly identifies the new last node. This differs from the singly linked list.

Insert at a position

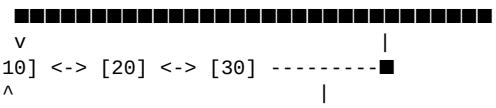
The lecture walks through inserting at a position by first traversing to the node before the insertion point, then updating the new node's next and previous references and repairing both neighboring links.

```
// Suppose trav is the node before the insertion point
Node after = trav.next;

newNode.previous = trav;
newNode.next = after;
trav.next = newNode;

if (after != null) {
    after.previous = newNode;
} else {
    tail = newNode; // insertion at the end
}
```

> Order matters: save the old next node before overwriting links. If you lose the reference to the successor, you may disconnect the rest of the list.

8. Circular doubly linked list

A circular doubly linked list combines two properties: each node has next and previous references, and the endpoints link back to each other. In a non-empty list, tail.next == head and head.previous == tail.



```
      ■------------------------■
Forward: 10 -> 20 -> 30 -> 10
Backward: 10 -> 30 -> 20 -> 10
```

Empty and single-node invariants

• Empty: head == null and tail == null.

• One node: head == tail; that node's next and previous point to itself.

• Multiple nodes: tail.next == head and head.previous == tail.

Add-first in a circular doubly linked list

```
Node newNode = new Node(value);

if (head == null) {
    head = newNode;
    tail = newNode;
    newNode.next = newNode;
    newNode.previous = newNode;
} else {
    newNode.next = head;
    newNode.previous = tail;
    head.previous = newNode;
    tail.next = newNode;
    head = newNode;
}
```

Add-last in a circular doubly linked list

```
Node newNode = new Node(value);

if (head == null) {
    head = newNode;
    tail = newNode;
    newNode.next = newNode;
    newNode.previous = newNode;
} else {
    newNode.previous = tail;
    newNode.next = head;
    tail.next = newNode;
    head.previous = newNode;
    tail = newNode;
}
```

Delete a node at a position: general pointer idea

The transcript demonstrates the need to connect the predecessor to the successor in both directions. If trav is the node to remove, its predecessor should skip it and point to its successor; the successor's previous should point back to the predecessor. Head and tail must be updated when the removed node is an endpoint.

```
Node before = trav.previous;
Node after = trav.next;

before.next = after;
after.previous = before;

// If removing head or tail, update those references too.
// For a one-node list, clear head and tail.
// For a circular list, preserve tail.next == head
// and head.previous == tail.
```

> The exact boundary branches depend on whether the list is linear or circular and whether it has zero, one, or multiple nodes. Do not copy the general-case pointer updates without checking these cases.

9. Implementation patterns and dry runs

Pointer update checklist

• Draw the list before changing any links.

• Identify the node being inserted or deleted and its neighbors.

• Store any next/previous references that would otherwise be overwritten.

• Update both directions for doubly linked lists.

• Update head and tail when the first or last node changes.

• Preserve the circular invariants when using a circular list.

• Test empty, one-node, two-node, and many-node lists.

Complexity summary

Operation	Singly list (head)	Singly list (head + tail)	Doubly list (head + tail)
Add first	O(1)	O(1)	O(1)
Add last	O(n)	O(1)	O(1)
Delete first	O(1)	O(1)	O(1)
Delete last	O(n)	O(n)	O(1)
Search by value	O(n)	O(n)	O(n)
Insert/delete at known node	Depends on access	Depends on access	O(1) pointer changes

Important distinction: the last row counts pointer changes when the target node (and required neighbors) are already known. Finding a node by value or position may still require O(n) traversal.

10. Common mistakes

• Calling temp.next when temp is null in the one-node delete-last case.

• Throwing an exception for an empty list but forgetting to return after handling a special case.

• Updating head but forgetting tail when a list becomes empty.

• Updating tail but forgetting tail.next or head.previous in a circular doubly linked list.

• Traversing a circular list until null, causing an infinite loop.

• Updating one direction of a doubly linked list but not the other.

• Assuming a tail pointer makes singly linked delete-last O(1).

• Claiming O(n/2) is a different Big-O class from O(n).

• Overwriting a link before saving the node that follows it.

11. Practice questions and answer key

Questions

• 1. Why does delete-last in a singly linked list need the second-last node?

• 2. What are the two key special cases for delete-last?

• 3. What is the complexity of add-last with only head? Why?

• 4. What does a tail pointer improve, and what does it not improve in a singly linked list?

• 5. What should the last node’s next reference contain in a circular singly linked list?

• 6. Why is do-while often convenient for circular traversal?

• 7. In a doubly linked list, if A.next = B, what should B.previous be?

• 8. What are the empty-list invariants for a head-and-tail implementation?

• 9. What are the circular doubly linked-list endpoint invariants?

• 10. If a question asks for insertion at position p, why might O(p) be a useful description?

• 11. What is the complexity of deleting the last node from a doubly linked list when tail is maintained?

• 12. Name four test cases to run after implementing a linked-list operation.

Answer key

• 1. A singly linked node has no previous reference, so the predecessor must be found by traversing from head.

• 2. Empty list (throw/handle an error) and one-node list (set head to null; also tail to null if maintained).

• 3. O(n), because the algorithm must traverse to the last node.

• 4. Tail makes add-last O(1); it does not provide the second-last node, so singly linked delete-last remains O(n).

• 5. It points to head, not null.

• 6. It executes the body once before checking whether traversal has returned to head; still guard against an empty list.

• 7. B.previous should refer to A.

• 8. head == null and tail == null.

• 9. tail.next == head and head.previous == tail; for one node, next and previous point to itself.

• 10. The traversal may visit nodes up to the requested position; relative to n, worst-case complexity is O(n).

• 11. O(1), because tail.previous directly identifies the new last node, assuming the structure is valid.

• 12. Empty list, one node, two nodes, and several nodes (including endpoint and middle operations).

12. Final revision checklist

• ■ I can draw and explain head, tail, next, and previous references.

• ■ I can implement delete-last for a singly linked list and handle empty and single-node cases.

• ■ I know the typical complexity of display, add-first, add-last, delete-first, and delete-last.

• ■ I can explain why tail improves add-last but not singly linked delete-last.

• ■ I can traverse a circular singly linked list without an infinite loop.

• ■ I can maintain both next and previous references in a doubly linked list.

• ■ I can preserve head/tail and circular endpoint invariants.

• ■ I can dry-run pointer changes on paper before coding.

• ■ I have tested empty, single-node, two-node, and multi-node cases.

Source and scope note

These notes were created from the supplied Lecture 3 transcript. The transcript includes live coding narration and repeated pointer demonstrations; some identifiers and lines are unclear in the automatic transcription. Therefore, code examples here are clean Java reconstructions that illustrate the described logic. The transcript's main focus is linked-list deletion and complexity, tail-pointer optimization, singly circular lists, doubly linked lists, and circular doubly linked lists. A full comparison of all six sorting algorithms was mentioned as future material, not taught in this lecture.